Point-Supervised Remote-Sensing Segmentation

Partial Cross-Entropy, controlled experiment design, and evidence audit

1 November 2026

Submission status: engineering package prepared; empirical assessment incomplete. This report distinguishes current verification from planned experiments and unrecovered historical evidence. No segmentation-performance result is claimed.

1. Abstract

Semantic segmentation assigns a class to each image pixel, but dense annotation of large aerial scenes is expensive. Point supervision reduces the number of labels used for training while leaving most pixels unlabelled. This project implements Partial Cross-Entropy (PCE), integrates it with a U-Net using a ResNet-18 encoder, and specifies controlled LandCover.ai experiments on annotation density and spatial sampling. The reconstructed implementation passes **131 tests, with one CUDA-only test skipped**. Its five-class network has **14,328,789 parameters**; verified ImageNet initialization and a tested CPU dependency lock are available. These are software measurements, not segmentation results. The four required experimental conditions have **zero completed real-data runs**: the official dataset, original split manifest and trained checkpoints were not recovered, and the archive download remains blocked by the environment proxy. Consequently, mIoU, Dice, per-class quality, seed variability and qualitative predictions are unavailable. The contribution demonstrated here is an audited, reproducible implementation and experimental protocol. The intended empirical comparison remains a mandatory completion requirement.

2. Introduction

Remote-sensing semantic segmentation supports land-cover mapping by identifying buildings, woodland, water, roads and background in aerial imagery. Dense masks encode object boundaries and small structures but require extensive annotation and quality control. Roads may be narrow, buildings may be occluded, and land-cover boundaries may be ambiguous. A small set of labelled pixels offers a different supervision budget: only the selected locations contribute directly to the loss.

The assessment asks for three outcomes: implement PCE; generate simulated point labels from real remote-sensing segmentation data and integrate them with a network; and investigate one or two performance factors in a technical report. The supplied one-page assessment was read, including its embedded problem statement. Exact requirements and the document hash appear in `docs/assessment_requirements.md` and `docs/assessment_source.json`.

This study proposes two questions: how performance changes from 10 to 50 to 200 points per training tile, and whether spatial stratification improves performance at a fixed 50-point budget. These questions concern segmentation quality under controlled supervision. They do not establish annotation labour savings, which would require observations of human annotation time and accuracy.

3. Related background

Fully supervised segmentation learns from dense per-pixel targets. U-Net [2] combines an encoder with a decoder and skip connections that retain spatial information. Residual networks [3] provide an established encoder family; ImageNet pretraining offers a useful RGB initialization, although natural images and aerial scenes differ in scale, appearance and viewing geometry.

Weak supervision uses less complete labels, including points, scribbles or image-level tags. Point-supervised segmentation has been studied explicitly [4]. PCE is the cross-entropy objective restricted to labelled pixels. Unlabelled pixels have no direct logit gradient from this objective, but predictions remain coupled through shared convolutional parameters and image context. The present implementation uses no pseudo-labels, additional consistency loss, class-guided point selection or ensemble. The assessment's introductory framework illustration is background; its explicit PCE task and the user's loss contract govern this implementation. In particular, no focal reweighting is added.

4. Dataset and preprocessing

The intended source is **LandCover.ai v1**, original aerial RGB GeoTIFFs and dense semantic masks [1]. The user-supplied recovery checkpoint records 41 scenes: 33 at 0.25 m/pixel and eight at 0.5 m/pixel, with three uint8 image bands and one integer label band. The recorded license is CC BY-NC-SA 4.0; imagery attribution is the Head Office of Geodesy and Cartography, Poland. These dataset facts are historical source information, **not an inventory independently repeated in the current environment**. The repository's MIT license covers its code, not the dataset imagery or annotations.

Index	Meaning	Treatment
0	Background	Valid training and evaluation class
1	Building	Valid class
2	Woodland	Valid class
3	Water	Valid class
4	Road	Valid class
255	Unselected location or padding	Excluded from supervision; padding excluded from evaluation

Original scenes are grouped **before tiling**. An edge connects scenes whose footprints are at most 256 m apart; identical image hashes also create edges. Connected components are indivisible split groups. Source CRS definitions are preserved, including WKT without an EPSG authority. Only footprint geometry is transformed to EPSG:2180 for distance calculations; image and mask pixels are not reprojected or resampled. Sorted groups are permuted with PCG64 and seed 2026. The reconstructed allocator uses floor(0.70G) training groups, round(0.15G) validation groups and the remainder for test, where G is the group count. Audits check disjoint scene IDs, group IDs, image hashes and cross-split distances.

The historical frozen manifest was lost. Its reported assignments and distances cannot certify a newly generated split. The reconstructed algorithm is documented, but historical byte identity is not claimed. A future manifest must be audited, frozen and retained before training. This custom geographic protocol differs from the official

prepared 512-pixel benchmark split. Geographic separation reduces one leakage risk; it does not prove independence of acquisition conditions.

Native-resolution, nonoverlapping 256×256 windows use stride 256. Edge padding is invalid. RGB becomes float32 divided by 255, then normalized with mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. Masks are never interpolated. Capped subsets are selected without class information, using subset seed 2718 independently of condition and run seed. Validation and test use complete masks without augmentation. Their availability on the real dataset remains unverified here.

5. Methodology

5.1. Network and sparse annotations

The network is the Segmentation Models PyTorch [6] U-Net with ResNet-18 encoder, RGB input, decoder widths $(256, 128, 64, 32, 16)$ and five raw output logits. Non-multiples of 32 are padded before the network and cropped afterward. The parameter count is 14,328,789, verified by tests and the initialization record. The official ResNet-18 ImageNet weights were downloaded and their full SHA-256 verified. Requested initialization failure raises an error; checkpoint loading does not download initialization weights. The architecture remains unchanged from the reconstructed Phase 3 implementation.

Uniform sampling permutes valid coordinates with a local NumPy PCG64 generator and takes a prefix. With the same tile/run seed, 10-, 50- and 200-point sets are nested. Stratified sampling constructs exactly K cells using horizontal bands and column divisions, draws from valid pixels in each nonempty cell, and fills empty-cell deficits uniformly from remaining coordinates. Both methods select $\min(K, V)$ distinct points, where V is the tile's valid pixel count. Neither method uses hidden semantic labels. Seeds derive from run seed, scene ID, tile ID and a strategy namespace using the documented reconstructed SHA-256 scheme.

Coordinates are fixed before augmentation and never refreshed between epochs. Joint D4 rotations/reflections preserve alignment of image, sparse targets, point mask and validity mask. Sparse training batches return label 255 everywhere unselected; dense references require an explicit diagnostic interface or the separately configured dense reference. Reading dense source masks to simulate points does not expose those unselected targets to the sparse loss.

5.2. Partial Cross-Entropy

For the set M of valid annotated pixel indices across the whole batch,

$$L_{\mathrm{PCE}} = -\frac{1}{N} \sum_{i \in M} \log p_\theta(y_i \mid x_i), \qquad N = |M|.$$

Here x_i denotes the image context for pixel i, $y_i \in \{0, \dots, C-1\}$ is its selected class, $C = 5$, θ denotes network parameters and $p_\theta(y_i \mid x_i)$ is the softmax probability of that class. A location belongs to M only if its point mask is true, its target is not 255 and its class index is in range. The logit implementation is $\mathrm{logsumexp}_c\, z_{ic} - z_{i,y_i}$, averaged over all N selected pixels. This avoids unequal image weighting when annotation counts differ.

Selection occurs **before** loss arithmetic, so unselected NaNs cannot contaminate the objective. Float16/bfloat16 arithmetic is promoted to float32. For $N = 0$, the loss returns an autograd-connected zero; the trainer skips forward propagation and optimizer updates, preserving BatchNorm buffers and AdamW state. Tests cover

fully labelled CE equivalence, single points, empty selections, invalid labels, batch weighting, numerical stability and selected/unselected gradients.

5.3. Optimization, evaluation and provenance

Training uses AdamW, count-weighted epoch loss, and the configured cosine or constant schedule. Python, NumPy and Torch seeds are recorded; deterministic Torch algorithms and a separate loader generator are used. Checkpoints contain model, optimizer, scheduler, scaler and RNG state, configuration, source/subset hashes and history. Resume is at epoch boundaries with identical controls, source, runtime, output directory and preserved best/last artifacts. This supports repeat execution on the same environment; it does not promise bitwise equality across hardware or library versions.

One global confusion matrix is accumulated over all valid evaluation pixels, with truth rows and prediction columns. For class c, IoU is $TP_c/(TP_c + FP_c + FN_c)$ and Dice is $2TP_c/(2TP_c + FP_c + FN_c)$. Macro averages include background. Classes absent from truth and predictions are undefined and omitted; false-positive-only classes score zero and remain included. Pixel accuracy is total correct pixels divided by all valid pixels. Dense CE, support and the full matrix accompany scalar scores.

Selection uses maximum dense validation mIoU, with the earliest epoch winning an exact tie. Training constructs only training and validation datasets. Test inference requires explicit access and frozen checkpoint/configuration/manifest membership. The final review added score-to-confusion consistency checks, full-history selection validation, checkpoint hash checks and validation of cached final-test metrics before reuse. These checks establish artifact consistency, not proof against deliberately fabricated source data.

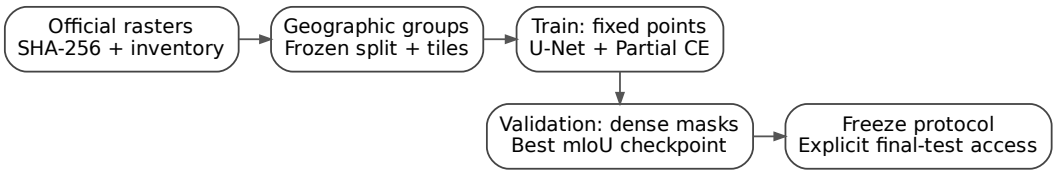


Figure 1: Implemented workflow. Solid paths describe code, not completed real-data execution. Validation selects checkpoints; test access follows protocol freezing.

6. Experimental design

Experiment A: increasing point density is hypothesized to improve accuracy, potentially with diminishing returns. The independent variable is 10, 50 or 200 uniformly sampled points per tile. **Experiment B:** spatial stratification is hypothesized to improve geographic coverage and possibly prediction consistency at 50 points. A2 serves as B1. Coverage alone would not demonstrate higher mIoU.

ID	Sampling	Points/tile	Question
A1	Uniform	10	Low annotation budget
A2/B1	Uniform	50	Shared density/strategy comparison
A3	Uniform	200	Higher annotation budget
B2	Stratified	50	Spatial distribution at equal budget

Within a campaign, architecture, initialization policy, split, preprocessing, subsets, augmentation, optimizer, scheduler, batch size, horizon and evaluation must remain fixed. Paired seeds control model initialization, shuffling and augmentation. Sampling randomness remains explicit and condition-dependent. The optional dense reference uses ordinary CE and all valid labels; it is an upper reference in supervision availability, not a guaranteed accuracy bound. It is not scheduled ahead of the required sparse comparisons.

The candidate `cpu_paired` profile specifies 10 epochs, batch two, no loader workers, seeds 17/29/43, and train/validation/test tile caps 128/32/32. AdamW uses learning rate 10^{-3} and weight decay 10^{-4}; the cosine minimum is 10^{-6} and early stopping is disabled. It is a separate profile; the original main and CPU-pilot configurations were preserved. Available compute is four CPU cores, 32 GiB memory and no CUDA device. One training process with four Torch threads is planned. At the caps there are at most 640 training batches per run; this workload count is not a runtime estimate. Real-data timing is still needed to establish how many paired seed blocks are feasible.

For a fully valid tile, nominal fractions are 0.01525879%, 0.07629395% and 0.30517578% for 10, 50 and 200 points. These are arithmetic values, not measured campaign densities. Actual fractions must divide unique fixed annotations by pooled valid training pixels, accounting for edge tiles and clipped budgets. Coverage diagnostics use occupied cells in a fixed 8×8 grid and normalized nearest-neighbour distance, without consulting semantic labels.

The runner provides named runs, immutable configuration snapshots, separate directories, checkpoint/resume, structured metrics, failures and completed-run protection. Priority is one run per required condition, then additional paired seeds, then the optional dense reference. Final test access follows freezing. Means and sample SDs require completed repeated runs; one seed yields a single value with no invented SD. Small differences with few seeds would not establish statistical significance.

7. Experimental results and verified evidence

7.1. Real-data execution status

No Phase 4 real-data campaign run or test evaluation completed. Twelve candidate runs (four conditions by three seeds) are marked `blocked_prerequisites` in `results/cpu_paired/run_status.csv`. `results.json` contains an empty list and `results.csv` contains only its header. The data preparation retry again failed with proxy CONNECT HTTP 403. A saved network configuration draft has not activated the dataset hostname in the applied recovery environment policy.

ID	Completed seeds	mIoU	Mean Dice	Pixel accuracy	Status
A1	0	—	—	—	Blocked
A2/B1	0	—	—	—	Blocked
A3	0	—	—	—	Blocked
B2	0	—	—	—	Blocked
Dense, optional	0	—	—	—	Not scheduled

A dash means **unmeasured**, not zero. Per-class IoU, training curves, actual annotation fractions, training duration, qualitative predictions and error maps are unavailable. Consequently, no empirical performance chart is included. The workflow figure illustrates implemented code only. Notebook geometric plots are labelled synthetic

and are not offered as remote-sensing results. Historical smoke numbers from the failed environment are excluded from all result tables.

7.2. Current software verification

Evidence	Verified outcome	Scope
E01: current suite	131 passed; 1 skipped; 0 failures/errors	Synthetic inputs and raster fixtures
E02: locked install	Fresh CPU environment install and forward pass	77 pinned packages; artifact hashes
E03: model/initialization	14,328,789 parameters; verified weight SHA-256	Architecture and initialization
E05: result export	0 measured rows	Campaign artifact inventory
E06: real-data gate	Unsatisfied	Archive, split and real smoke missing
E07: notebook	Sequential execution, explicit missing-data notices	No campaign training or test inference

Tests exercise real U-Net tensor behavior, sparse raster loaders, gradient isolation, empty-batch protection, metric indexing, split constraints, checkpoint/RNG restoration and deterministic resume. Fast runner tests also use a small convolutional model. These generated fixtures test contracts; they do not validate real-world segmentation. CUDA training and AMP are unverified.

The machine-readable evidence inventory maps these claim IDs to source files and SHA-256 fingerprints. It also records each of the twelve run IDs, its configuration, status and absent metric files. The uploaded assessment hash is recorded as E10. PDF rendering and archive integrity have separate validation records; neither changes the empirical completion status.

8. Discussion

Hypothesis outcomes. Neither density nor stratification is supported or rejected by current measurements: both are untested. More labelled points add supervision under the proposed protocol, but monotonic measured improvement cannot be inferred from label count alone. Likewise, spreading points across space does not ensure rare semantic classes are sampled or that accuracy improves.

Class imbalance and failure analysis. Actual class supports and confusion matrices are absent. It would be speculative to identify the best or worst class in this implementation. Once runs exist, supports, false positives, false negatives and per-class IoU should be inspected together. Pixel accuracy can be dominated by common classes; macro IoU helps expose weak classes but can be variable when classes are scarce. The absent-class convention must accompany any comparison. Narrow objects, mixed boundaries, shadows and complex scenes are plausible stressors to examine, not observed failure cases here.

Uncertainty and generalization. Three paired seeds would quantify some optimization and annotation-sampling variation on a fixed split, not uncertainty over all possible regions or acquisition campaigns. Small CPU subsets may omit important terrain or classes. Geographic grouping cannot establish full statistical independence, and a custom split does not support direct leaderboard comparison. Natural-image

initialization has a domain mismatch. Fixed points avoid accumulating additional hidden supervision across epochs, but synthetic uniform points differ from human choices, mistakes and uncertainty.

Useful supervision and next evidence. No measured annotation threshold can be recommended. A useful-accuracy threshold requires an application-specific criterion, actual held-out metrics and an annotation-cost model. Completion requires verified official rasters and partitions, real-data smoke/reload, controlled runs, a frozen protocol and held-out evaluation. Training/validation curves and preselected qualitative tiles should then support explanations of optimization stability and errors. The optional dense reference should remain secondary to finishing the four required sparse conditions.

9. Conclusion

The current package demonstrates a tested Partial Cross-Entropy implementation, an integrated segmentation pipeline on synthetic fixtures, explicit experimental controls and reproducibility safeguards. It does **not** yet demonstrate point supervision on real LandCover.ai data or measured effects of annotation density and sampling strategy. The original assessment's Task 1 is verified; Task 2's real-data demonstration and Task 3's experimental results remain incomplete. The appropriate next step is to restore data access and execute the existing protocol, retaining actual manifests, checkpoints, metrics and plots. The submission should not be represented as a completed empirical assessment until those requirements are verified.

10. References

1. A. Boguszewski et al. "LandCover.ai: Dataset for Automatic Mapping of Buildings, Woodlands, Water and Roads from Aerial Imagery." *CVPR Workshops*, 2021, pp. 1102–1110. Dataset homepage.
2. O. Ronneberger, P. Fischer and T. Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation." *MICCAI*, 2015, pp. 234–241. arXiv:1505.04597.
3. K. He, X. Zhang, S. Ren and J. Sun. "Deep Residual Learning for Image Recognition." *CVPR*, 2016, pp. 770–778. arXiv:1512.03385.
4. A. Bearman, O. Russakovsky, V. Ferrari and L. Fei-Fei. "What's the Point: Semantic Segmentation with Point Supervision." *ECCV*, 2016, pp. 549–565. arXiv:1506.02106.
5. A. Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *NeurIPS*, vol. 32, 2019. arXiv:1912.01703.
6. P. Iakubovskii. *Segmentation Models Pytorch*. Software, 2019; version 0.6.0 used here. Official repository.

Bibliographic links identify the cited scientific sources; full papers were not retrieved in this restricted environment. ResNet/SMP repository metadata was cross-checked. Dataset provenance and license were retained from the supplied recovery checkpoint and are flagged accordingly in `report/reference_checks.json` and `docs/source_evidence.json`. PyTorch [5], torchvision, NumPy, rasterio, Shapely, Matplotlib and notebook dependencies are versioned in the tested CPU lock.